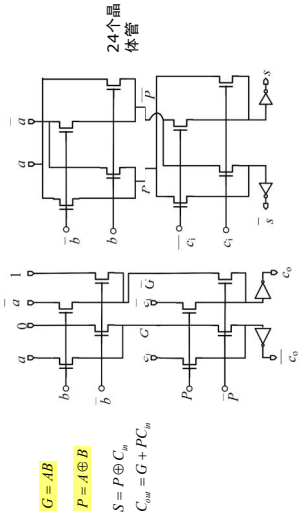


全加器电路（1）——CPL全加器

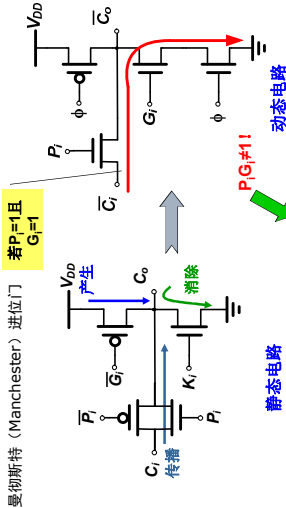


2023-2024秋

第10章 数据通路设计

8 清华大学 李刚宇

全加器电路（2）——曼彻斯特进位门

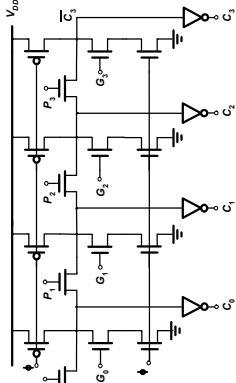


2023-2024秋

第10章 数据通路设计

9 清华大学 李刚宇

曼彻斯特 (Manchester) 进位链



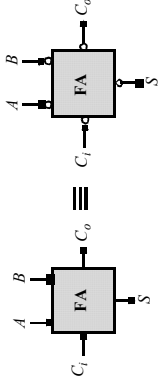
- 采用动态逻辑降低复杂性和加快速度。
- 进位链传输管只用N管，节点电容很小，为四个扩散电容。

2023-2024秋

第10章 数据通路设计

10 清华大学 李刚宇

反相特性



$$\bar{S}(A, B, C_i) = S(\bar{A}, \bar{B}, \bar{C}_i)$$
$$\bar{C}_o(A, B, C_i) = C_o(\bar{A}, \bar{B}, \bar{C}_i)$$

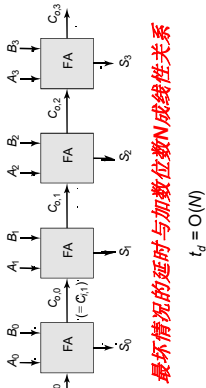
2023-2024秋

第10章 数据通路设计

5 清华大学 李刚宇

N位加法器

行波进位 (Ripple-Carry) 加法器



最坏情况的延时与加数位N成线性关系

$$t_d = O(N)$$

$$t_{adder} = (N-1)t_{carry} + t_{sum}$$

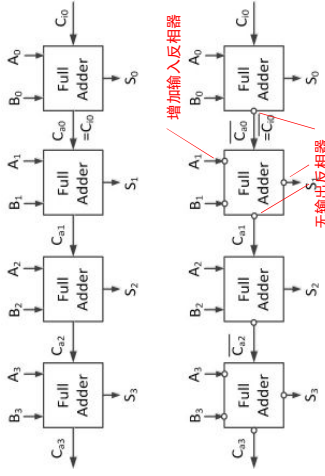
关键：加快进位信号传播的速度

2023-2024秋

第10章 数据通路设计

6 清华大学 李刚宇

消除反相器的行波进位加法器



增加输入反相器

无输出反相器

2023-2024秋

第10章 数据通路设计

7 清华大学 李刚宇

加法器设计

二进制加法器

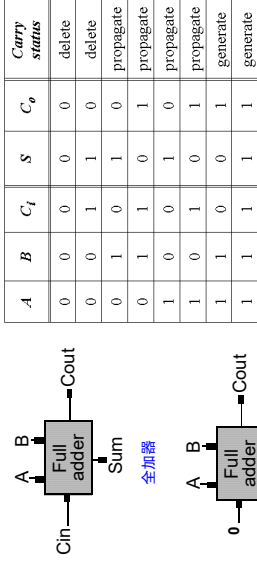
- 改进串行进位加法器
- 进位选择加法器
- 超前进位加法器
- 树形加法器

2023-2024秋

第10章 数据通路设计

2 清华大学 李刚宇

二进制加法



$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A + B) \quad (\text{major运算})$$

2023-2024秋

第10章 数据通路设计

3 清华大学 李刚宇

进位产生、进位取消、进位传播信号

- 进位产生信号 (G)
- 进位消除信号 (K或D)
- 进位传播信号 (P)

$$G = AB$$

$$K = \overline{AB}$$

$$P = A \oplus B$$

$$S(G, P) = P \oplus C_i$$

$$C_o(G, P) = G + PC_i$$

也可以取 $A+B$

$$C_o = AB + BC_i + AC_i$$

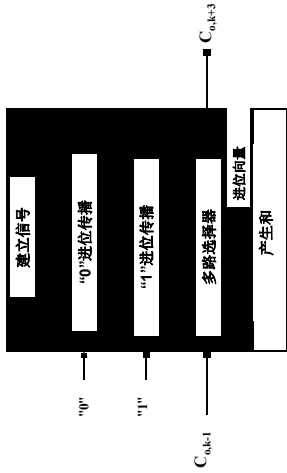
$$S = ABC_i + \overline{C_o}(A + B + C_i)$$

2023-2024秋

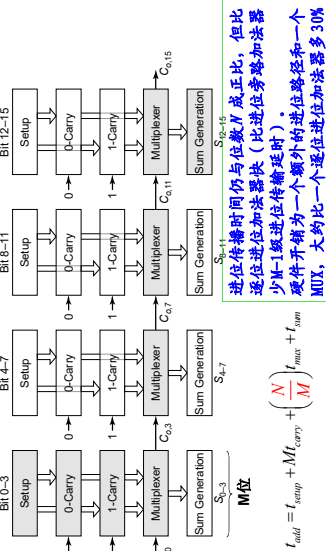
第10章 数据通路设计

4 清华大学 李刚宇

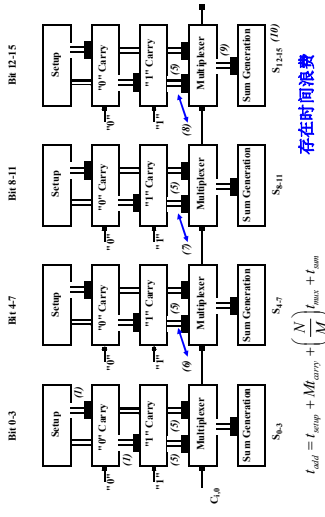
进位选择加法器



进位选择加法器: 关键路径



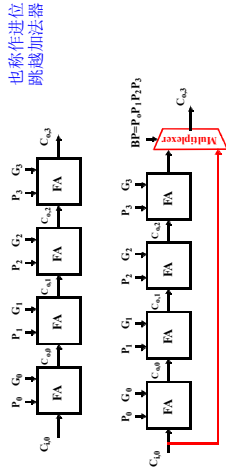
线性进位选择加法器



加法器设计

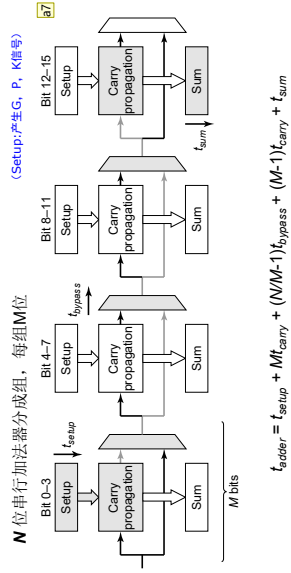
- 二进制定法器
- 改进串行进位加法器
- 进位选择加法器
- 超前进位加法器
- 树形加法器

进位旁路(Carry-bypass)加法器



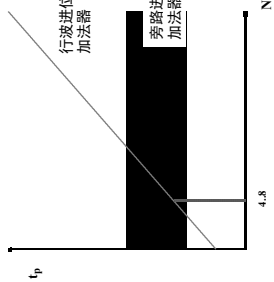
原理: 如果 (P_0, P_1, P_2, P_3) 均 $= 1$,
 $[Co_3 = C_{i,0}]$ 则主路徑進位消除或進位產生。

进位旁路加法器的关键路径



$$t_{\text{adder}} = t_{\text{setup}} + Mt_{\text{carry}} + (N/M-1)t_{\text{bypass}} + (M-1)t_{\text{carry}} + t_{\text{sum}}$$

行波进位与进位旁路比较



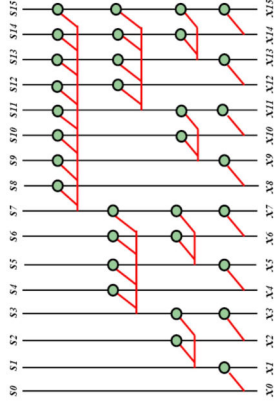
- 增加进位旁路一般使面积增加10%至20%。
·进位传播时间仍与位数 N 成正比，但比例系数比逐位进位加法器为小。
· N 较小时，旁路的额外开销比采用旁路进位的收益不大。
·一般旁路进位和行波进位的交叉点位于 N 在4~8之间的范围。

加法器设计

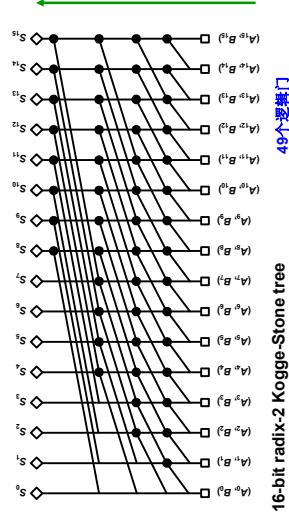
- ❏ 二进制加法器
 - ❏ 改进串行进位加法器
 - ❏ 进位选择加法器
 - ❏ 超前进位加法器
 - ❏ 树形加法器

Sklansky 加法器

把16位加法分解成两个8位的进位计算，高8位的进位由自身的子prefix结果和7：0位的块建立信号产生，……依次递归。



Kogge-Stone树形加法器



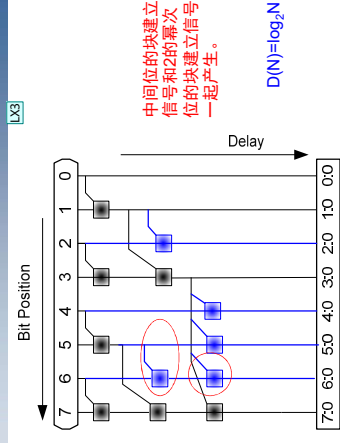
Kogge-stone加法器的特点

- 逻辑层数正比于 $\log_2 N$ 因此传播延时也正比于 $\log_2 N$
 - 结构规则，关键路径扇出较小而且基本是一个常数，确定尺寸容易
 - 版图紧凑（一定程度上可以消除门数较多带来的面积开销）
 - 门数较多
 - 各级之间有很多长连线
- 广泛应用于高性能的32位加法器和64位加法器。

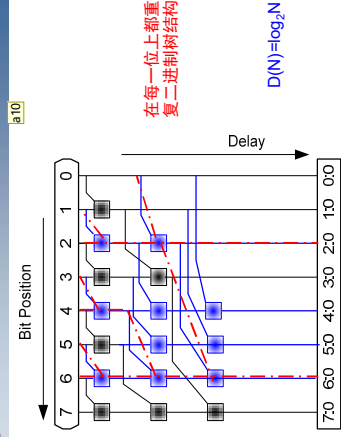
Brent-Kung树的特点

- 结构简单、门数少(面积小、功耗低)
- 关键路径长, 最多(S_{14})经过6级基本进位算子
- 结构不规则, 门的尺寸确定困难

Sklansky树结构



Kogge-Stone树结构



二进制乘法器

部分积产生

部分积的累加

最终相加

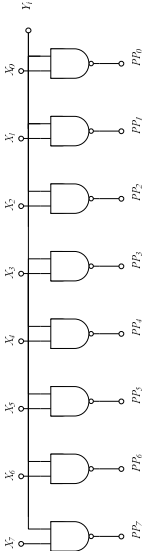
高性能乘法器结构

部分积产生

部分积是被乘数X和一个乘数位Y_i相乘的结果，一位二进制乘法由AND实现。

部分积产生逻辑

- 共N个部分积，MXN个AND（NAND）



部分积压缩——波兹编码

乘数中等于0的位所对应的部分积对结果没有影响，因此可以只计算非零位的部分积，减少部分积的数目

通过编码的方法可以减少非零行的数目

- 改进的波兹编码（Booth）：每次乘数中取k位（例如k=2）与被乘数相乘产生部分积，并在乘数编码中引入-1来减少非零位。

$$2^{i+k-1} + 2^{i+k-2} + \dots + 2^i = 2^i (2^k - 1) = \underbrace{2^i}_{\text{相当于k位全为1}} \times (2^k - 1)$$

预计算后的有符号数乘法

modified Baugh-Wooley 乘法器

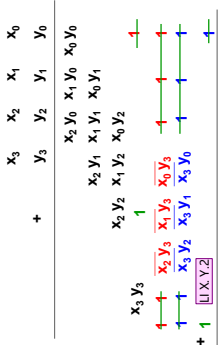
$$\begin{array}{r} x_3 \ x_2 \ x_1 \ x_0 \\ + \quad y_3 \ y_2 \ y_1 \ y_0 \\ \hline 1 \quad \overline{x_0 y_3} \quad x_2 y_0 \quad x_1 y_0 \quad x_0 y_0 \\ \quad \overline{x_1 y_3} \quad x_2 y_1 \quad x_1 y_1 \quad x_0 y_1 \\ \quad \quad \overline{x_2 y_3} \quad x_2 y_2 \quad x_1 y_2 \quad x_0 y_2 \\ \quad \quad \quad 1 \quad x_3 y_3 \quad \overline{x_3 y_2} \quad \overline{x_3 y_1} \quad \overline{x_3 y_0} \\ \hline S_6 \ S_5 \ S_4 \ S_3 \ S_2 \ S_1 \ S_0 \end{array}$$

与无符号数乘法器有相似的部分积阵列，设计时可以通过把无符号数乘法器中某些部分积的位用NAND计算，把部分积累加时的闲置输入端置为1实现。因此，有符号数乘法和无符号数乘法可以用相同的阵列结构来实现。

把上一阵的部分积重新排列得到如圈所示的阵列排列。注意：相同权重（带下标）的部分积是可以交换

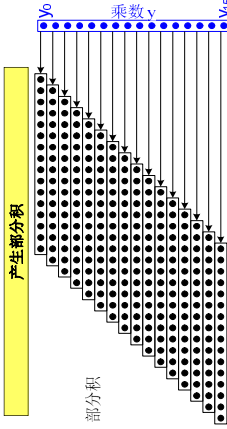
有符号数的二进制乘法

$$X = -x_{m+1}2^{m+1} + \sum_{i=m+2}^{m+2} x_i 2^i, Y = -y_{n+1}2^{n+1} + \sum_{i=n+2}^{n+2} y_i 2^i \quad \text{符号位 (MSB) 有负的权重}$$
$$XY = x_{m+1}y_{n+1}2^{m+n+2} + \sum_{i=m+2}^{m+2} x_i y_i 2^{m+n+2} - (x_{m+1} \sum_{i=n+2}^{n+2} y_i 2^{m+i+1} + y_{n+1} \sum_{i=m+2}^{m+2} x_i 2^{n+i+1})$$



乘法器结构

每个点代表部分积的一个比特

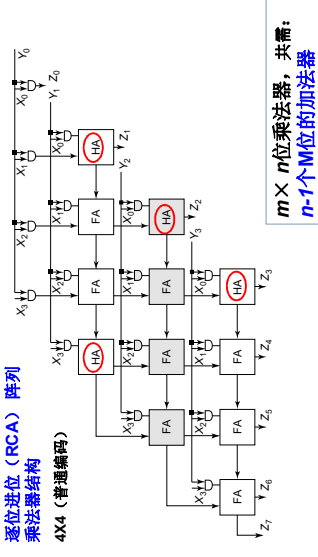


阵列乘法器实现：

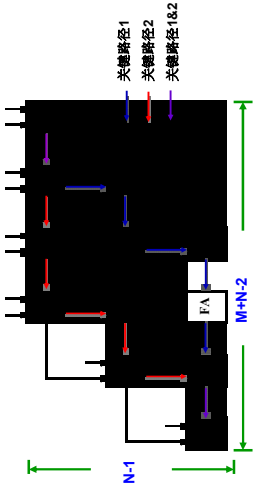
部分积产生-部分积累加-最终相加

为了确定溢出位的结果，在m+1位二进制补码表示“范围”内，对于负数加上2^{m+1}（m+1）后所得结果，此时得到的乘积就是溢出位的结果。其值等于1，则表示溢出；否则，则表示溢出位的结果为0。

阵列乘法器



MxN RCA阵列乘法器的关键路径



$$t_{mult} = [(M-1) + (N-2)]t_{carry} + (N-1)t_{sum} + t_{and}$$

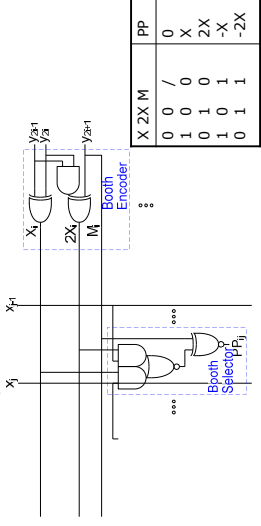
多条长度相同的关键路径： t_{sum} 和 t_{carry} 的延时间等重要

乘法器

- 二进制乘法器
- 部分积产生
- 部分积的累加
 - 逐位进位阵列乘法器
 - 进位保留乘法器
- 树形乘法器
- 最终相加
- 高性能乘法器结构

Booth编码器及部分积选择器

波兹编码的部分积计算需要编码、译码、移位和(求反+1)



改进Booth编码乘法的特点

- 优点：
 - 部分积数目减为原来的一半。可用于无符号数及2S补码数。
 - 代价是：部分积除0，X外还有+2X，-2X，+X，-X，因此需要译码。对负项还要“求反加1”。
- 在部分积累加时每产生一个部分积要左移2位
- 也可以同时考查4位，每3位进行一次编码(基8波兹编码)

乘法器

- 二进制乘法器
- 部分积产生
- 部分积的累加
 - 逐位进位阵列乘法器
 - 进位保留乘法器
- 树形乘法器
- 最终相加
- 高性能乘法器结构

改进的波兹编码

通过同时观察3位乘数进一步简化

即：在乘数的低位上加一“考察位”：考察 $y_{i+1}y_iy_{i-1}$ ，其中 y_{i-1} 为“考察位”。下次考察时考察 $y_{i+2}y_{i+1}y_i$ ，其中 y_i 为“考察位”。相邻两次考察时重叠一位乘数(y_{i+1})。

对于有符号数(2's补码表示)的乘数Y

$$Y = -y_{n-1}2^{n-1} + \sum_{j=0}^{n-2} y_j 2^j$$
$$= \sum_{i=0}^{i=n/2-1} (y_{2i-1} + y_{2i} - 2y_{2i+1})2^{2i}, y_{-1} = 0$$

编码公式

{2, -1, 0, 1, 2}

改进波兹编码的证明

$$X \cdot Y = (y_{n-1}2^{n-1} + y_{n-2}2^{n-2} + \dots + y_12^1 + y_02^0) \cdot X$$
$$= \sum_{i=0}^{n-1} (y_{i-1} + y_i - 2y_{i+1}) \cdot X \cdot 2^i$$
$$= (y_{-1} + y_0 - 2y_1) \cdot X \cdot 2^0 + (y_1 + y_2 - 2y_3) \cdot X \cdot 2^1 + \dots + (y_{n-2} + y_{n-1} - 2y_n) \cdot X \cdot 2^{n-1} = X \cdot Y$$

(若n为偶数)

(n为奇数)

改进波兹编码乘法

举例：X = 1101, Y = 1001, 4位乘4位 = 8位

将X、Y看成是无符号的正数，则 X = 13, Y = 9, X·Y = 117

乘数 Y =

1	0	0	1	0
				0
				0
				0
				0

注意：-2X为左移，求反，加1(成求反，左移)，左移。

左移时右边添0。

乘数 Y =

1	0	0	1	0
				0
				0
				0
				0

将X、Y看成2's补码(负数)，则 X = -3, Y = -7, X·Y = 21

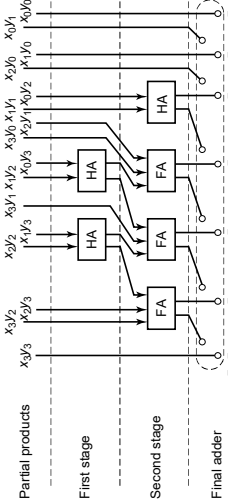
乘数 Y =

1	0	0	1	0
				0
				0
				0
				0

计算与无符号数相同

$y_{2i-1} + y_{2i} - 2y_{2i+1}$

Wallace-Tree 乘法器



Wallace Tree的特点

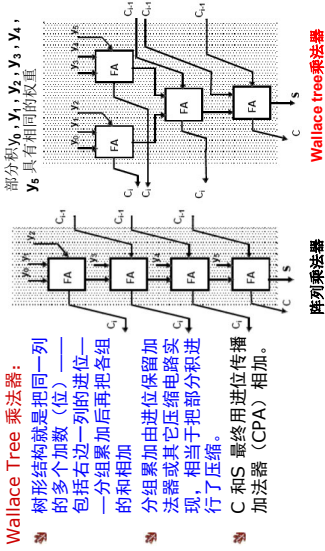
- 优点：
 - 在通常的阵列乘法器中，乘法延迟时间正比于 n 。
 - 在基4 改进波兹编码乘法器中，乘法延迟时间正比于 $n/2$ 。
 - 采用压缩比为3:2的Wallace Tree 乘法器的乘法延迟时间正比于 $\log_{3/2}(\frac{n}{2})$ 。最后还需将最终的两个数相加。
- 证明：有 n 个“部分积”，把 n 个“部分积”分成三个“部分积”为一组（共 $n/3$ 组）同时相加，每组相加的结果为两个数，共产生 $2(n/3)$ 个数。也就是 n 个“部分积”每加一次变为 $(2/3)n$ 个数，若经 x 次变为 2 个数。则 $x = \log_{3/2}(\frac{n}{2})$

缺点：Wallace Tree 乘法器非常不规则，给版图设计带来困难

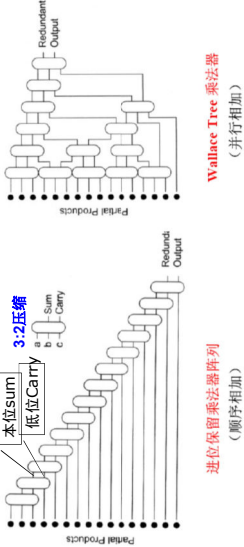
乘法器

- 二进制乘法器
- 部分积产生
- 部分积的累加
- 最终相加
- 高性能乘法器结构

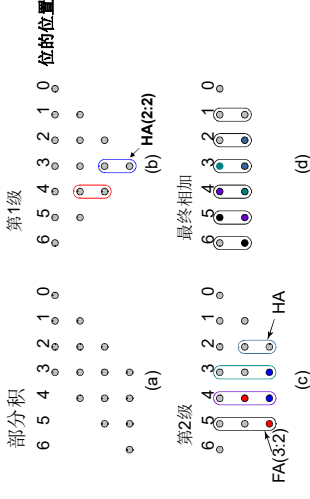
树形结构部分积累加



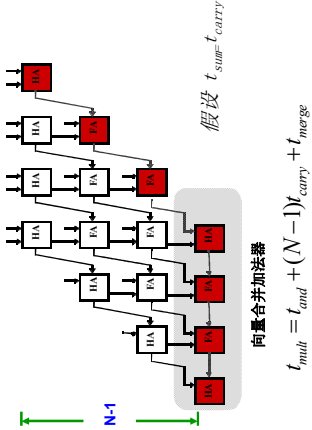
Wallace Tree 乘法器与阵列乘法器的比较



Wallace-Tree的压缩步骤



进位保留乘法器结构



进位保留加法器

- 一个 n 位的进位保留加法器 (Carry-Save Adder, CSA) : 由 n 个不相关的一位全加器构成，在这些全加器之间没有进位关系，即在一个 n 位 CSA 加法器的内部没有 Carry 传播。它的输入是要被相加的三个 n 位数，它的输出是两个字：
 - 一个字是 n 位的“和”： $S(S_{n-1}, \dots, S_1, S_0)$
 - 另一个字是 n 位的“进位”： $C(C_{n-1}, \dots, C_1, C_0)$ S 和 C 可以被送入另一个 n 位的 CSA 与第四个 n 位的数 W 相加；一般地，多个 n 位的数可以通过 CSA 树结构相加而产生。为得到最后的和， S 和 C 必须最终用一个带进位传播的通常的加法器 (CPA) 相加 (最终相加)。
- 在 CSA 中为了加快连续加法的速度，减少门延时，可以采用正码和反码两种 CSA，在阵列中交叉使用。
- 进位保留乘法器在最坏情形下的关键路径是唯一确定的。

乘法器

- 二进制乘法器
- 部分积产生
- 部分积的累加
 - 逐位进位阵列乘法器
 - 进位保留乘法器
 - 树形乘法器
- 最终相加
- 高性能乘法器结构

乘法器小结

- 第一次系统级的优化
- 优化目标和二进制加法器不同
- 又一次出现**关键路径**的概念：识别关键路径至关重要
- 其它可能的优化技术
 - 对数结构-线性结构(高压缩比的Wallace Tree)
 - Booth编码（高基波兹编码）
 - 流水技术

2023-2024秋

第10章 数据通路设计

80

清华大学 李刚宇

总结

- 数据通路的特点：
 - 位片式
 - 主要优化指标是性能和功耗，结构的规则性也非常重要
- 加法器：关键路径是进位传播路径
- 乘法器：
 - 总延时 = 部分积时间 + 部分积累加时间 + 最终相加时间
 - 可以通过波兹编码减少部分积数量

2023-2024秋

第10章 数据通路设计

81

清华大学 李刚宇

总结（续）

- 数据通路的优化方法
 - 线性结构的旁路：进位旁路加法器、超前进位加法器、进位保留
 - 先计算后选择：进位选择
 - 分组并行处理：树形加法器、树形乘法器
 - 流水线
 - 把关键路径上的操作移到非关键路径上：利用加法器的反相特性、平方根进位选择……
 - 预计算：在设计阶段完成常数运算，简化电路（乘法器的符号扩展位精简、波兹编码）

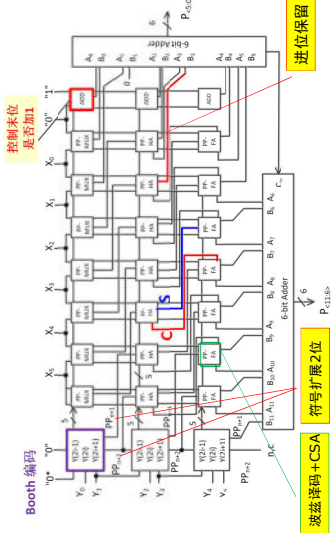
2023-2024秋

第10章 数据通路设计

82

清华大学 李刚宇

6X6 Booth编码CSA乘法器



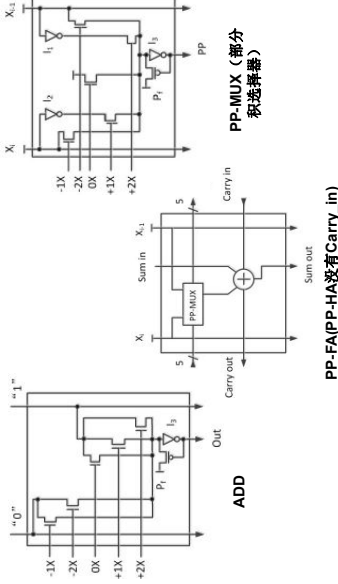
2023-2024秋

第10章 数据通路设计

77

清华大学 李刚宇

单元电路



2023-2024秋

第10章 数据通路设计

78

清华大学 李刚宇

基本单元功能描述

- ADD: 产生每个部分积的末位，如果编码为-2X或-X，则输出1，其输出在合并加法器中与累加结果相加
- PP-MUX: 按照编码结果选择部分积输出
- PP-FA (PP-HA): 包括两部分
 - PP-MUX
 - 加法器：把前几级的S、本地的PP、上级的Carry_out相加，产生S与Carry_out

2023-2024秋

第10章 数据通路设计

79

清华大学 李刚宇

最终相加

- 实现CSA或者Wallace Tree最后的向量合并
- 由进位传播加法器（CPA）实现，结构根据部分积累加阵列的结构而定
 - 如果所有输入同时到达(如流水线结构)，一般使用超前进位加法器
 - 输入到达时间不同，一般使用进位选择加法器，可以节省面积

2023-2024秋

第10章 数据通路设计

74

清华大学 李刚宇

乘法器

- 二进制乘法器
- 部分积产生
- 部分积的累加
- 最终相加
- 高性能乘法器结构

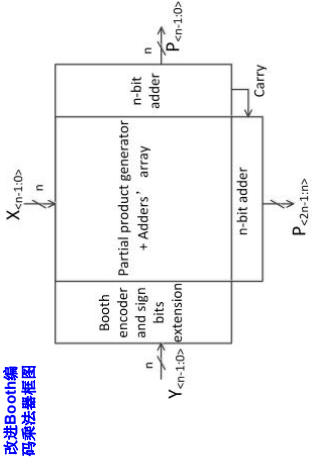
2023-2024秋

第10章 数据通路设计

75

清华大学 李刚宇

Booth编码CSA乘法器



2023-2024秋

第10章 数据通路设计

76

清华大学 李刚宇